

CODEX IMPLEMENTATION PLAN

Zombie Survival 2.5D Task Breakdown

Atomic, ordered tasks for AI-assisted implementation (Codex / Cursor / Copilot). Each task is self-contained with clear inputs, outputs, acceptance criteria, and file paths.

Based on GDD V1.3 · May 2026 · Godot 4.5 · GDScript

Overview

This document breaks the GDD into ordered, atomic implementation tasks designed for Codex. Each task has: ID, title, description, files to create/modify, acceptance criteria, and dependencies. Tasks are ordered so each builds on the last — implement them one by one in sequence.

Phase Summary

PHASE		TASKS	FOCUS	EST.
P0 — Bug Fixes	CRITICAL	5	Fix known bugs in current codebase	1 day
P1 — Code Health	REFACTOR	5	Split god objects, normalize style	3–4 days
P2 — Data Layer	DATA	5	New JSON schemas, data loading, validation	2–3 days
P3 — Hero Skills	NEW	6	Skill system, cooldowns, UI, hero kits	1–2 weeks
P4 — Weapon Tiers	NEW	4	Rarity, upgrades, special effects	1 week
P5 — Pet Refactor	REFACTOR	4	Remove attack code, add buff system	1 week
P6 — Guardian MVP	NEW	7	Bruiser Guard: follow, skills, AI, hire	3–4 days
P7 — In-Run Upgrades	NEW	3	Upgrade pool, selection UI, apply logic	3–4 days
P8 — Economy	NEW	4	Currencies, shop, energy, daily rewards	1–2 weeks

Total: 43 tasks • Est. 6—9 weeks (solo)

How to use with Codex: Feed each task as a prompt. Include the task description, file paths, acceptance criteria, and relevant context from previous tasks. Codex should produce a PR-ready diff for each task.

Dependency Graph

TASK CAN START AFTER...	PHASE
P0 (Bug Fixes) — no dependencies	Start immediately
P1 (Code Health) — after P0	Requires bug-free base
P2 (Data Layer) — after P0	Can parallel with P1
P3 (Hero Skills) — after P1 + P2	Needs clean code + data files
P4 (Weapon Tiers) — after P2	Needs data layer
P5 (Pet Refactor) — after P1 + P2	Needs clean code + data files
P6 (Guardian MVP) — after P2	Needs data layer; can parallel with P3—P5
P7 (In-Run Upgrades) — after P2	Needs data layer
P8 (Economy) — after P4 + P5	Needs weapon + pet systems

P0 — Bug Fixes CRITICAL

Fix known bugs before adding anything new. All are in existing files.

P0-01

~15 min

Fix Hero HP double-apply bug

`apply_selected_loadout()` in `game.gd` calls `player.apply_hero_definition()` which applies `max_hp_bonus`. Then `apply_selected_loadout()` applies it again. Knight gets +8 HP instead of +4.

- Remove the duplicate HP bonus application in `game.gd` → `apply_selected_loadout()`
- Keep only the application inside `player.apply_hero_definition()`

Files: `scripts/game.gd`, `scripts/player.gd`

ACCEPTANCE

- Knight starts with exactly 12 HP (base 8 + bonus 4), not 16
- All other heroes' HP matches their `max_hp_bonus` value in `heroes.json`

P0-02

~10 min

Fix PetCompanion attacking dead enemies

`pet_companion.gd` → `_find_nearest_enemy()` does not filter out dead enemies. Pet continues shooting at corpses.

- Add `if enemy.is_dead: continue` check in `_find_nearest_enemy()`

Files: `scripts/pet_companion.gd`

ACCEPTANCE

- Pet never targets an enemy with `is_dead == true`
- Pet smoothly retargets to next alive enemy

P0-03

~10 min

Fix camera shake never resetting

Camera shake offset accumulates and never returns to zero. After heavy combat the camera drifts.

- Add `offset = offset.lerp(Vector2.ZERO, delta * 8.0)` in the camera's `_process()`
- Or reset offset to `Vector2.ZERO` when shake timer expires

Files: `scripts/camera.gd` (or wherever camera shake is implemented)

ACCEPTANCE

- Camera returns to center after shake ends
- No permanent drift after extended gameplay

P0-04

~5 min

Set TEST_UNLOCK_ALL_FEATURES to false

`TEST_UNLOCK_ALL_FEATURES := true` is hardcoded. This bypasses all progression locks.

- Set `TEST_UNLOCK_ALL_FEATURES := false`
- Consider moving to a debug config or export flag

Files: `scripts/GameManager.gd` (or wherever the flag is defined)

ACCEPTANCE

- Default build has progression locks active
- Debug flag is easily toggleable for testing

P0-05

~5 min

Remove temp debug file + fix WaveManager dead enemy skip

`inspect_weapon_attach_tmp.gd` is committed. Also, WaveManager doesn't skip dead enemies during wave recycle, which can cause phantom enemies.

- Delete `inspect_weapon_attach_tmp.gd`
- In WaveManager recycle loop, add `if enemy.is_dead: continue`

Files: `scripts/inspect_weapon_attach_tmp.gd` (delete), `scripts/WaveManager.gd`

ACCEPTANCE

- Temp file removed from repo
- Wave transitions don't carry over dead enemy references

P1 — Code Health

REFACTOR

Split god objects, normalize code style. This makes all future tasks cleaner.

P1-01

~2-3 hrs

Normalize indentation across all .gd files

Codebase has mixed indentation: 22 files use tabs, 13 use spaces. GDScript standard is tabs.

- Convert all .gd files to tabs
- Verify no broken indentation after conversion

Files: all `scripts/*.gd` files

ACCEPTANCE

- All .gd files use tab indentation consistently
- Game runs and all scenes load without errors

P1-02

~3-4 hrs

Extract WeaponVisuals from player.gd

`player.gd` is 944 lines. ~300 lines handle weapon attachment, positioning, and visual effects. Extract into a new `WeaponVisuals` component.

- Create `scripts/components/weapon_visuals.gd`
- Move weapon attach, position, rotation, muzzle flash, shell casing logic
- `Player.gd` calls `weapon_visuals.attach_weapon(weapon_data)`, `weapon_visuals.play_fire_effect()`
- Keep `player.gd` focused on movement, combat, HP, and state

Files: `scripts/player.gd` → `scripts/components/weapon_visuals.gd` (new)

ACCEPTANCE

- `player.gd` reduced by ~250—300 lines
- All weapon visuals work identically to before
- No regressions in weapon switching, firing, or visual effects

P1-03

~4-6 hrs

Split `home_screen.gd` into sub-controllers

`home_screen.gd` is 1,215 lines with 80+ `@onready` vars. Split into logical sub-controllers.

- Create `scripts/ui/hero_select_controller.gd` — hero selection, preview, stats display
- Create `scripts/ui/weapon_select_controller.gd` — weapon grid, rarity display, equip logic
- Create `scripts/ui/pet_select_controller.gd` — pet selection, preview
- Create `scripts/ui/settings_controller.gd` — settings panel, audio, controls
- `home_screen.gd` becomes a coordinator that delegates to sub-controllers

Files: `scripts/home_screen.gd` → 4 new files in `scripts/ui/`

ACCEPTANCE

- `home_screen.gd` under 300 lines
- All home screen functionality preserved
- Each sub-controller handles its own `@onready` vars and signals

P1-04

~1-2 hrs

Deduplicate `_find_nearest_enemy()`

`_find_nearest_enemy()` is duplicated in multiple files (`player.gd`, `pet_companion.gd`, possibly others). Extract to a shared utility.

- Create `scripts/utils/combat_utils.gd` (autoload or static class)
- Add `static func find_nearest_enemy(from_position: Vector3, enemies: Array, max_range: float) -> Node`
- Must skip dead enemies
- Replace all duplicate implementations

Files: `scripts/utils/combat_utils.gd` (new), update `player.gd`, `pet_companion.gd`

ACCEPTANCE

- Single source of truth for enemy targeting
- Dead enemies always skipped
- All callers produce identical behavior to before

P1-05

~1 hr

Clean up unused LevelDifficultyData fields

`GameData.gd` defines `LevelDifficultyData` with fields that are never read by game logic. Remove or wire them up.

- Audit which fields in `LevelDifficultyData` are actually consumed
- Remove unused fields, or add TODO comments for future use

Files: `scripts/GameData.gd`

ACCEPTANCE

- No dead-code fields without clear TODO annotations
- Data class is clean and documented

P2 — Data Layer DATA

Create new JSON data files and loading/validation logic. This is the foundation for all new systems.

P2-01

~1-2 hrs

Create skills.json — hero skill definitions

Define all hero skills with cooldowns, damage, effects, unlock levels. See GDD Section 2.2 for full skill trees.

- Create `data/skills.json`
- Schema per skill: `id`, `hero_id`, `name`, `type` (passive/active), `cooldown`, `damage`, `effect`, `description`, `unlock_level`
- Include all 15 skills (3 per hero × 5 heroes) from GDD Section 2.2

Files: `data/skills.json` (new)

ACCEPTANCE

- Valid JSON, parseable by Godot
- All 15 skills defined with complete fields
- Values match GDD Section 2.2

P2-02

~1 hr

Create guardians.json — guardian definitions

Define all 5 guardians and their skills. MVP only uses Bruiser Guard, but define all for future. See GDD Section 5.3—5.4.

- Create `data/guardians.json`
- Schema per guardian: `id`, `display_name`, `type`, `role`, `follow_distance`, `scan_interval`, `skills[]` (each with `name`, `cooldown`, `damage`, `range`, `shape`, `trigger_condition`, `effect`), `model_scene_path`, `rarity`, `unlock_condition`
- Bruiser Guard skills: Slam (5s CD, 2.5m, 1–2 dmg), Cleave (2.5s CD, short cone, 1 dmg), Emergency Heal (12–15s CD, trigger player HP < 40%, heal 1–2)

Files: `data/guardians.json` (new)

ACCEPTANCE

- Valid JSON with all 5 guardian kits defined
- Bruiser Guard values match GDD Section 5.3 exactly

P2-03

~1 hr

Create `pet_evolution.json` — pet evolution & buff data

Define per-stage buff multipliers for all 5 pets across 3 evolution stages. See GDD Section 4.2.

- Create `data/pet_evolution.json`
- Schema per pet: `pet_id`, `stages[]` (each with `stage`, `shard_cost`, `buff_multiplier`, `visual_change`)
- Include base buff values from GDD Section 4.1 pet roster

Files: `data/pet_evolution.json` (new)

ACCEPTANCE

- Valid JSON with all 5 pets × 3 stages
- Buff values match GDD Section 4.1–4.2

P2-04

~1 hr

Create `upgrades.json` — in-run upgrade pool

Define all in-run upgrades with tiers, effects, and weights. See GDD Section 6.

- Create `data/upgrades.json`
- Schema per upgrade: `id`, `name`, `tier` (common/rare/epic), `effect_type`, `effect_value`, `description`, `icon`, `weight`, `max_stack`
- Include full upgrade pool from GDD Section 6.1

Files: `data/upgrades.json` (new)

ACCEPTANCE

- Valid JSON with all upgrades from GDD Section 6
- Tier weights sum correctly for selection probability

Add data loading & validation to GameData.gd

Extend GameData.gd to load and validate all new JSON files at startup with fallback defaults.

- Add `load_skills()`, `load_guardians()`, `load_pet_evolution()`, `load_upgrades()` methods
- Follow existing pattern: load JSON → validate required fields → fallback to defaults on error
- Add typed data classes: `SkillData`, `GuardianData`, `PetEvolutionData`, `UpgradeData`
- Add lookup helpers: `get_skill(id)`, `get_guardian(id)`, etc.

Files: `scripts/GameData.gd`

ACCEPTANCE

- All new JSON files load at startup without errors
- Invalid/missing data falls back gracefully (prints warning, uses defaults)
- Lookup helpers return correct data for any valid ID

P3 — Hero Skills NEW SYSTEM

Add the hero skill system: SkillManager, cooldown tracking, UI indicators, and individual hero skill implementations.

P3-01

~3-4 hrs

Create SkillManager component

Core component that manages active/passive skills, cooldown timers, and skill execution. Attaches to player node.

- Create `scripts/components/skill_manager.gd`
- `func load_skills(hero_id: String)` — reads from `skills.json` via GameData
- `func try_use_skill(skill_id: String) -> bool` — checks cooldown, executes if ready
- `func _process(delta)` — tick all cooldown timers
- Signals: `skill_activated(skill_id)`, `skill_cooldown_updated(skill_id, remaining)`
- Support both active skills (manually triggered or auto-triggered) and passives (always-on stat modifiers)

Files: `scripts/components/skill_manager.gd` (new)

ACCEPTANCE

- Can load skills for any hero from data
- Cooldowns tick correctly and block re-use
- Signals fire on activation and cooldown changes

P3-02

~2-3 hrs

Create skill cooldown UI

HUD elements showing skill icons with cooldown overlays. Mobile-friendly layout (bottom of screen).

- Create `scenes/ui/skill_hud.tscn` + `scripts/ui/skill_hud.gd`
- Display 2—3 skill slots with icon, cooldown sweep overlay, and ready indicator
- Connect to SkillManager signals
- Auto-hide passive skills (show only active skills with cooldowns)

Files: `scenes/ui/skill_hud.tscn` (new), `scripts/ui/skill_hud.gd` (new)

ACCEPTANCE

- Skill icons visible during gameplay
- Cooldown overlay animates smoothly
- Touch-friendly on mobile (min 48×48 px tap targets)

Implement Knight skills

Knight skill set from GDD: Iron Skin (passive, -1 damage), Shield Bash (active, 8s CD, cone knockback), War Cry (active, 14s CD, +30% team damage 6s).

- Create `scripts/skills/knight_skills.gd`
- Iron Skin: modify `player.take_damage()` to subtract 1 (min 1 damage)
- Shield Bash: cone detection → apply damage + knockback to enemies in cone
- War Cry: temporary buff → increase player + pet + guardian damage for 6s

Files: `scripts/skills/knight_skills.gd` (new), modify `player.gd`

ACCEPTANCE

- Iron Skin reduces damage by 1 (verified in combat)
- Shield Bash knocks back enemies in cone, 8s cooldown works
- War Cry buff applies and expires correctly after 6s

Implement Rogue skills

Rogue skill set from GDD: Shadow Step (passive, 12% dodge), Backstab (active, 6s CD, teleport behind + 3× damage), Smoke Bomb (active, 16s CD, 3s invisibility + slow enemies).

- Create `scripts/skills/rogue_skills.gd`
- Shadow Step: modify `player.take_damage()` → 12% chance to negate
- Backstab: teleport behind target + apply 3× damage + return
- Smoke Bomb: create area effect → player invisible + enemies in zone slowed

Files: `scripts/skills/rogue_skills.gd` (new)

ACCEPTANCE

- Dodge triggers ~12% of the time (test with 100+ hits)
- Backstab teleport feels snappy, damage multiplied correctly
- Smoke Bomb creates visible area, enemies slow, player untargetable for 3s

P3-05

~3-4 hrs

Implement Mage skills

Mage skill set from GDD: Arcane Amplify (passive, +15% projectile damage), Frost Nova (active, 10s CD, freeze AoE), Meteor Strike (active, 20s CD, large AoE).

- Create `scripts/skills/mage_skills.gd`
- Arcane Amplify: modify projectile damage calculation → +15%
- Frost Nova: AoE around player → freeze enemies 2s + damage
- Meteor Strike: delayed AoE at target position → heavy damage + burn

Files: `scripts/skills/mage_skills.gd` (new)

ACCEPTANCE

- Arcane Amplify applies consistent +15% to all projectile damage
- Frost Nova freezes enemies in radius for 2s
- Meteor Strike has visible delay, large damage, uses explosion VFX

P3-06

~1-2 hrs

Integrate SkillManager with player.gd and game.gd

Wire everything together: SkillManager loads on game start, player triggers skills, HUD updates.

- In `game.gd` : instantiate SkillManager, call `load_skills(selected_hero_id)`
- In `player.gd` : add reference to SkillManager, apply passive modifiers on load
- In game HUD: add SkillHud scene, connect to SkillManager signals
- Active skills auto-trigger based on conditions (or add skill buttons for mobile)

Files: `scripts/game.gd` , `scripts/player.gd` , game HUD scene

ACCEPTANCE

- Selecting Knight/Rogue/Mage loads their skills automatically
- Skills activate during gameplay and show on HUD
- Full gameplay loop works: skills + combat + waves

P4 — Weapon Tiers

NEW SYSTEM

Add rarity system, weapon upgrades, and special effects. Builds on existing weapons.json.

P4-01

~2-3 hrs

Add rarity tiers to weapons.json

Extend existing `weapons.json` with rarity fields and stat multipliers. GDD Section 3.1: Common (1.0×), Uncommon (1.15×), Rare (1.3×), Epic (1.5×), Legendary (1.8×).

- Add `rarity` field to each weapon entry
- Add `rarity_multipliers` global config or per-weapon overrides
- Update GameData weapon loading to apply rarity multiplier to base stats
- Add rarity color mapping: Common (grey), Uncommon (green), Rare (blue), Epic (purple), Legendary (orange)

Files: `data/weapons.json`, `scripts/GameData.gd`

ACCEPTANCE

- Each weapon has a rarity field
- Rare Pistol has 1.3× damage vs Common Pistol
- GameData returns correct modified stats

P4-02

~3-4 hrs

Implement weapon upgrade system

Gold-based weapon upgrade with level 1—10, +8% per level. GDD Section 3.4.

- Add `weapon_levels` dict to SaveManager (weapon_id → level)
- Create upgrade cost formula: `base_cost * level^1.5`
- Apply upgrade multiplier: `1.0 + (level - 1) * 0.08`
- Add UI: upgrade button in weapon select screen with cost display

Files: `scripts/SaveManager.gd`, `scripts/GameData.gd`, weapon select UI

ACCEPTANCE

- Can upgrade weapon from level 1 to 2 (costs gold, stats increase)
- Level persists across runs via SaveManager
- Upgrade reflected in actual combat damage

P4-03

~4-5 hrs

Add weapon special effects

Each weapon type gets a unique special effect from GDD Section 3.3 (e.g., Pistol = ricochet, Shotgun = stagger, SMG = overhear burst, Sniper = pierce, Launcher = cluster bombs, Crossbow = poison).

- Create `scripts/components/weapon_effects.gd`
- Implement each effect as a method triggered on hit or on fire
- Effects activate based on weapon type + rarity (higher rarity = stronger effect)
- Use existing VFX where possible

Files: `scripts/components/weapon_effects.gd` (new), modify projectile system

ACCEPTANCE

- Each weapon type triggers its unique special effect
- Effects scale with rarity
- No performance issues on mobile

P4-04

~2-3 hrs

Update weapon UI with rarity visuals

Weapon selection and inventory should show rarity with color borders, star ratings, and stat comparisons.

- Color-code weapon cards by rarity
- Show rarity label and star count
- Show stat comparison (base vs upgraded vs rarity-modified)
- Add special effect description text

Files: weapon select UI scene + script

ACCEPTANCE

- Weapon cards show correct rarity color
- Stats displayed match actual gameplay values
- Special effect described clearly

P5 — Pet Refactor

REFACTOR

Transform pet from active attacker to passive support. Remove attack code, add buff/aura system.

P5-01

~1-2 hrs

Remove pet attack code

Currently `pet_companion.gd` has `_find_nearest_enemy()`, projectile firing, and attack timer. Remove all active combat logic.

- Remove `_find_nearest_enemy()` from `pet_companion.gd`
- Remove projectile spawning and attack timer
- Remove attack-related vars (`attack_range`, `attack_interval`, `attack_damage`)
- Keep: follow logic, visual positioning, animation

Files: `scripts/pet_companion.gd`

ACCEPTANCE

- Pet follows player but never fires at enemies
- No errors or null references from removed code
- Pet visually present and animated

P5-02

~3-4 hrs

Create BuffProvider component for pets

New component that reads pet buff data and emits stat modifiers to player and guardian. See GDD Section 4.1 for buff roster.

- Create `scripts/components/buff_provider.gd`
- Reads pet buff data from `pet_evolutions.json` via `GameData`
- Provides `get_active_buffs() -> Dictionary` (`buff_type` → value)
- Buff types: `damage_bonus`, `move_speed_bonus`, `gold_bonus`, `max_hp_bonus`, `regen_hp`
- Buffs scale with evolution stage
- Player reads buffs via `pet.buff_provider.get_active_buffs()`

Files: `scripts/components/buff_provider.gd` (new), modify `pet_companion.gd`

ACCEPTANCE

- Drone pet gives player +10% damage (visible in actual combat numbers)
- Sprite pet gives +8% move speed (player moves faster)
- Buffs scale correctly with evolution stage

P5-03

~2-3 hrs

Add pet evolution UI

UI to evolve pets using evolution shards. Show current stage, next stage buffs, and shard cost.

- Add evolution panel to pet select screen
- Show: current stage (1/2/3), current buffs, next stage buffs, shard cost
- Evolve button: spend shards → increment stage → update visual + buffs
- Add `evolution_stage` and `evolution_shards` to SaveManager

Files: pet select UI, `scripts/SaveManager.gd`

ACCEPTANCE

- Can evolve a pet from stage 1 → 2 (costs shards, buffs improve)
- Evolution persists across sessions
- Visual preview shows upgraded pet appearance

P5-04

~2 hrs

Add pet equipment / accessories

Simple accessory slot that amplifies the pet's base buff. GDD Section 4.3.

- Add `accessory_id` to pet save data
- Define accessories in JSON: `id`, `name`, `buff_amplify_percent`, `special_effect`
- BuffProvider applies accessory modifier on top of evolution buffs
- Simple equip UI in pet select screen

Files: `data/pet_accessories.json` (new), `scripts/components/buff_provider.gd`, pet UI

ACCEPTANCE

- Equipping an accessory visibly increases pet buff values
- Accessory persists in save data

P6 — Guardian MVP: Bruiser Guard

NEW SYSTEM

Implement the Bruiser Guard from GDD Section 5. Follow player, use 3 cooldown-based melee skills, invulnerable in MVP.

Key constraints: No projectiles. Scan every 0.2s (not per frame). Reuse existing VFX. Guard DPS $\leq$ 30% of player. Invulnerable (no HP). Cleanup on game over.

P6-01

~2-3 hrs

Create BruiserGuard scene and base script

New CharacterBody3D scene for the Bruiser Guard with basic structure and follow behavior.

- Create `scenes/entities/guardians/bruiser_guard.tscn`
- Create `scripts/entities/guardian/bruiser_guard.gd`
- Node structure: CharacterBody3D → CollisionShape3D, MeshInstance3D (placeholder model), AnimationPlayer (placeholder)
- Load stats from `guardians.json` via GameData
- Basic `_process()` : move toward player position with offset
- Follow distance: ~2.0 units behind/beside player
- Simple movement: `position = position.lerp(target_pos, delta * follow_speed)`

Files: `scenes/entities/guardians/bruiser_guard.tscn` (new), `scripts/entities/guardian/bruiser_guard.gd` (new)

ACCEPTANCE

- Guard spawns near player
- Guard follows player smoothly at short distance
- Guard does not block player movement
- Guard does not jitter or overshoot

Implement Guard AI state machine

4-state loop: Follow → EvaluateSkill → CastSkill → Cooldown. See GDD Section 5.5.

- Add `enum State { FOLLOW, EVALUATE, CAST, COOLDOWN }`
- FOLLOW: move toward player, transition to EVALUATE every 0.2s
- EVALUATE: scan for enemies, check cooldowns, pick skill (priority order from GDD 5.5.3)
- CAST: play skill (short duration), apply effects, transition to COOLDOWN
- COOLDOWN: return to FOLLOW immediately (individual skill timers track cooldowns)
- Use `scan_timer` (0.2s interval) — NOT `_process()` per frame

Files: `scripts/entities/guardian/bruiser_guard.gd`

ACCEPTANCE

- Guard evaluates skills every 0.2s (verify with debug print)
- State transitions are clean — no stuck states
- Guard returns to following when no skill conditions met

Implement Slam Knockback skill

Guard's primary skill. AoE around guard, pushes enemies away. GDD Section 5.3.

- Trigger: 2+ enemies within 2.5m of guard/player AND cooldown ready
- Cooldown: 5s
- Damage: 1—2 (use `randi_range(1, 2)`)
- Effect: iterate all enemies in radius → apply damage → apply velocity impulse outward from guard center
- VFX: reuse existing explosion/shockwave effect (scale down) + camera shake (light, 0.1s)
- Optional: light hit stun (0.2s enemy freeze) if stun system exists

Files: `scripts/entities/guardian/bruiser_guard.gd`

ACCEPTANCE

- Slam activates when 2+ enemies near
- Enemies visibly pushed away from guard
- Damage numbers appear (1 or 2)
- Camera shakes lightly
- 5s cooldown enforced

Implement Short Cleave skill

Frontal arc attack. Guard's primary damage source. GDD Section 5.3.

- Trigger: enemy within short range (~1.5m) in front of guard AND cooldown ready
- Cooldown: 2.5s
- Damage: 1
- Shape: ~90° cone in guard's facing direction
- Hits multiple enemies in cone
- VFX: reuse hit spark effect on each enemy hit + small slash arc effect if available

Files: `scripts/entities/guardian/bruiser_guard.gd`

ACCEPTANCE

- Cleave hits enemies in frontal cone
- Multiple enemies take 1 damage each
- Hit spark on each hit
- 2.5s cooldown enforced
- Does NOT hit enemies behind guard

Implement Emergency Heal skill

Auto-trigger heal when player HP is low. GDD Section 5.3.

- Trigger: player HP < 40% of max HP AND cooldown ready
- Cooldown: 12–15s (use 12s default)
- Heal: `randi_range(1, 2)` HP
- VFX: green pulse / flash on player position. Reuse existing heal effect or create simple particle
- Cannot trigger if player at full HP
- Highest priority in skill selection (checked first)

Files: `scripts/entities/guardian/bruiser_guard.gd`

ACCEPTANCE

- Heal triggers automatically when player HP drops below 40%
- Player HP increases by 1–2
- Green visual effect visible
- 12s cooldown enforced — no spam
- Does NOT trigger when player HP > 40%

Integrate guardian spawning and lifecycle

Wire guardian into game loop: spawn, follow, cleanup.

- Add `selected_guardian_id` and `active_guardian_ref` to GameManager
- In `game.gd`: spawn guardian when run starts (if hired)
- Position guardian near player at spawn
- On game over / scene exit: `active_guardian_ref.queue_free()`
- Guardian reference available to other systems (pet buff targeting, etc.)

Files: `scripts/GameManager.gd`, `scripts/game.gd`

ACCEPTANCE

- Guardian spawns at run start if hired
- Guardian removed cleanly on game over
- No orphaned nodes or errors on scene transition
- Can start run without guardian (guardian optional)

Add "Hire Guard" action

Simple UI to hire Bruiser Guard before/during run. Placeholder for future rewarded ads.

- Add "Hire Guard" button on home screen (near pet/weapon selection)
- On tap: set `GameManager.selected_guardian_id = "bruiser"`
- Add placeholder function: `request_hire_guard(guard_id)`
- Add placeholder hook: `on_rewarded_ad_completed_hire_guard(guard_id)`
- For now: hiring is free (debug mode). Future: require ad watch or gem spend.

Files: home screen UI, `scripts/GameManager.gd`

ACCEPTANCE

- "Hire Guard" button visible on home screen
- Tapping it → guardian spawns in next run
- Can start run without tapping (no guardian)
- Placeholder hooks exist for ad integration

P7 — In-Run Upgrades NEW SYSTEM

Add the roguelite upgrade selection system. Player picks from 3 options between waves.

P7-01

~2-3 hrs

Create UpgradeManager

Core system that manages the upgrade pool, selection, and application during a run.

- Create `scripts/systems/upgrade_manager.gd`
- Load upgrade pool from `upgrades.json` via `GameData`
- `func roll_upgrades(count: int = 3) -> Array` — select random upgrades weighted by tier
- `func apply_upgrade(upgrade_id: String)` — apply effect to player/pet/guardian
- Track applied upgrades for stacking limits
- Tier weights: Common 60%, Rare 30%, Epic 10%

Files: `scripts/systems/upgrade_manager.gd` (new)

ACCEPTANCE

- Rolling 3 upgrades returns valid, distinct choices
- Tier distribution roughly matches weights over many rolls
- Stacking limits enforced

P7-02

~3-4 hrs

Create upgrade selection UI

Popup between waves showing 3 upgrade cards. Player picks one. Game pauses during selection.

- Create `scenes/ui/upgrade_selection.tscn` + `scripts/ui/upgrade_selection.gd`
- 3 cards showing: icon, name, tier (color-coded), description, current stack count
- Tap card → apply upgrade → close popup → resume game
- Timer: auto-pick random if no choice in 10s (or skip)
- Trigger popup after every N waves (configurable, default: every 3 waves)

Files: `scenes/ui/upgrade_selection.tscn` (new), `scripts/ui/upgrade_selection.gd` (new)

ACCEPTANCE

- Popup appears between waves
- 3 cards visible with correct data
- Tapping a card applies the upgrade and resumes game
- Game pauses while popup is visible

Implement upgrade effects

Wire each upgrade type to actual stat changes on player, pet, guardian, and weapons.

- Effect types: `damage_flat`, `damage_percent`, `fire_rate`, `move_speed`, `max_hp`, `heal`, `xp_magnet_range`, `gold_bonus`, `cooldown_reduction`
- Each effect modifies the appropriate stat on the relevant system
- Upgrades reset at end of run (not persistent)
- Stack tracking: some upgrades cap at `max_stack` (e.g., `damage_flat` caps at 5)

Files: `scripts/systems/upgrade_manager.gd`, `scripts/player.gd`

ACCEPTANCE

- Picking "+1 Damage" upgrade increases actual combat damage by 1
- Picking "Move Speed +10%" makes player noticeably faster
- All upgrades reset when run ends

P8 — Economy & Engagement

NEW SYSTEM

Add currencies, shop, energy system, and daily engagement features.

P8-01

~3-4 hrs

Implement currency system

3 currencies: Gold (earned in-run), Gems (premium/rare), Evolution Shards (pet-specific). See GDD Section 7.1.

- Add currency tracking to SaveManager: `gold`, `gems`, `shards` (per pet)
- Gold earned during gameplay (enemy drops, wave bonuses)
- Gems earned rarely (boss kills, achievements, daily rewards)
- Shards earned from runs with pet equipped
- Add `add_currency(type, amount)` and `spend_currency(type, amount) -> bool`
- Show currency in HUD (gold during run) and home screen (all currencies)

Files: `scripts/SaveManager.gd`, HUD, home screen UI

ACCEPTANCE

- Gold increases during gameplay (visible in HUD)
- Currencies persist across sessions
- Cannot spend more than available

P8-02

~2-3 hrs

Implement energy system

5 energy max, 1 per run, regenerates 1 every 10 minutes. GDD Section 7.2.

- Add energy tracking to SaveManager: `energy`, `last_energy_time`
- Regeneration: calculate earned energy since last check on app open
- Starting a run costs 1 energy
- Block run start if energy = 0 (show timer to next energy)
- Future: watch ad to get +1 energy, gems to refill

Files: `scripts/SaveManager.gd`, `scripts/GameManager.gd`, home screen UI

ACCEPTANCE

- Energy decreases by 1 when starting a run
- Energy regenerates over time (verify by changing system clock)
- Cannot start run with 0 energy

Create shop UI

Basic shop for spending gold on weapon upgrades, hero leveling, and pet evolution. Placeholder for IAP integration.

- Create `scenes/ui/shop.tscn` + `scripts/ui/shop.gd`
- Tabs: Weapons (upgrade), Heroes (level up), Pets (evolve), Gems (IAP placeholder)
- Each item shows: cost, current level, benefit preview
- Purchase flow: tap → confirm → deduct currency → apply upgrade

Files: `scenes/ui/shop.tscn` (new), `scripts/ui/shop.gd` (new)

ACCEPTANCE

- Shop accessible from home screen
- Can purchase weapon upgrade with gold
- Insufficient gold shows error/disabled state

Add daily rewards and quests

Simple daily login reward + 3 daily quests for engagement. GDD Section 7.5.

- Daily login: escalating rewards over 7 days (gold → gems → shards → premium item)
- Track `last_login_date` and `login_streak` in SaveManager
- 3 daily quests: "Kill 50 zombies", "Complete 3 runs", "Use 5 skills" (random from pool)
- Quest tracking during gameplay, reward on completion
- Reset quests daily

Files: `scripts/systems/daily_rewards.gd` (new), `scripts/SaveManager.gd`, home screen UI

ACCEPTANCE

- Daily reward popup on first login each day
- Streak increases on consecutive days
- Quest progress tracked during runs
- Quest rewards granted on completion

Appendix: Quick Reference

Existing Codebase Files (as of latest review)

FILE	LINES	ROLE
scripts/GameManager.gd	651	Global state, settings, hero/weapon/pet selection
scripts/player.gd	944	Player entity (movement, combat, weapon, HP) — split in P1
scripts/GameData.gd	398	Data loading, validation, typed data classes
scripts/SaveManager.gd	146	Persistence (JSON save/load)
scripts/game.gd	~300	Main game scene controller
scripts/enemy.gd	377	Enemy entity (movement, HP, drops)
scripts/WaveManager.gd	223	Wave spawning and progression
scripts/pet_companion.gd	110	Pet entity (follow + attack → refactor to buff)
scripts/home_screen.gd	1,215	Home screen UI — split in P1
scripts/projectile.gd	~50	Bullet entity (pooled)

New Files Created by This Plan

PHASE	NEW FILE	PURPOSE
P1	scripts/components/weapon_visuals.gd	Extracted weapon visual logic
P1	scripts/ui/hero_select_controller.gd	Home screen sub-controller
P1	scripts/ui/weapon_select_controller.gd	Home screen sub-controller
P1	scripts/ui/pet_select_controller.gd	Home screen sub-controller
P1	scripts/ui/settings_controller.gd	Home screen sub-controller

PHASE	NEW FILE	PURPOSE
P1	<code>scripts/utils/combat_utils.gd</code>	Shared targeting utility
P2	<code>data/skills.json</code>	Hero skill definitions
P2	<code>data/guardians.json</code>	Guardian definitions
P2	<code>data/pet_evolutions.json</code>	Pet evolution data
P2	<code>data/upgrades.json</code>	In-run upgrade pool
P3	<code>scripts/components/skill_manager.gd</code>	Skill system core
P3	<code>scenes/ui/skill_hud.tscn</code>	Skill cooldown HUD
P3	<code>scripts/skills/knight_skills.gd</code>	Knight skill implementations
P3	<code>scripts/skills/rogue_skills.gd</code>	Rogue skill implementations
P3	<code>scripts/skills/mage_skills.gd</code>	Mage skill implementations
P4	<code>scripts/components/weapon_effects.gd</code>	Weapon special effects
P5	<code>scripts/components/buff_provider.gd</code>	Pet buff system
P5	<code>data/pet_accessories.json</code>	Pet equipment data
P6	<code>scenes/entities/guardians/bruiser_guard.tscn</code>	Bruiser Guard scene
P6	<code>scripts/entities/guardian/bruiser_guard.gd</code>	Bruiser Guard script
P7	<code>scripts/systems/upgrade_manager.gd</code>	In-run upgrade system
P7	<code>scenes/ui/upgrade_selection.tscn</code>	Upgrade selection popup
P8	<code>scenes/ui/shop.tscn</code>	Shop UI
P8	<code>scripts/systems/daily_rewards.gd</code>	Daily rewards system

Codex Prompt Template

Use this template when feeding tasks to Codex:

```
## Task: [TASK_ID] – [TASK_TITLE]

### Context
- Project: Zombie Survival 2.5D (Godot 4.5, GDScript)
- Repo: https://github.com/tinhtinhcd/zombie2.5d
- This task is part of Phase [X] of the implementation plan.
- Previous completed tasks: [list completed task IDs]

### Description
[Copy task description and bullet points]

### Files to Create/Modify
[Copy file paths]

### Acceptance Criteria
[Copy acceptance criteria]

### Reference
[Include relevant GDD section text or existing code snippets]
```

This plan is based on GDD V1.3 (May 2026). Task estimates assume one developer using Codex/AI assistance. Adjust order if priorities change — but always complete PO first.